Kiến thức Machine Learning tổng quát

Tổng hợp từ tài liệu scikit-learn, các bài giảng về bias-variance, các bài so sánh metric.

1. Kiến thức nền chung

- **Ba loại học máy**: có giám sát (supervised), không giám sát (unsupervised), tăng cường (reinforcement). Người mới thường bắt đầu với supervised, ví dụ bài toán **phân loại nhị phân**.
- **Phân loại vs hồi quy**: dự đoán nhãn rời rạc (thu nhập >50K hay không) vs dự đoán số liên tục (giá nhà).
- **Thuật ngữ**: feature (X), label/target (y), tham số (model tự học, ví dụ trọng số) vs siêu tham số (người chọn trước, ví dụ `max_depth`, `n_estimators`).
- **Hàm mất mát (loss) và tối ưu**: model học bằng cách giảm loss, thường qua gradient descent. Chỉ cần hiểu ý tưởng: đo sai bao nhiêu rồi chỉnh tham số để sai ít đi.
- **Quy trình chuẩn**: hiểu bài toán → khám phá dữ liệu (EDA) → tiền xử lý → chia dữ liệu → huấn luyện → đánh giá → tinh chỉnh → báo cáo.
- **Tiền xử lý cơ bản**: xử lý giá trị thiếu, mã hóa biến phân loại (one-hot), chuẩn hóa thang đo (scaling).

2. Kiến thức quan trọng (phần hay bị hổng nhất)

Generalization (khả năng tổng quát hóa). Mục tiêu không phải điểm cao trên dữ liệu đã thấy, mà là dự đoán tốt trên dữ liệu mới.

Bias–Variance, underfitting, overfitting. Khi model linh hoạt hơn thì bias giảm nhưng variance tăng. Underfitting là bias cao (model quá đơn giản). Overfitting là variance cao (model học cả nhiễu). Dấu hiệu điển hình của overfitting là điểm train cao nhưng điểm test thấp hơn nhiều.

Chia dữ liệu: train / validation / test. Khi tinh chỉnh siêu tham số nên chia ba phần (ví dụ 60/20/20): train để học, validation để chỉnh, test chỉ để báo cáo cuối. Test set là đề thi thật, chỉ dùng một lần.

Cross-validation (CV). Một lần chia train/test có thể không đại diện. K-fold CV chia dữ liệu thành K phần, lần lượt dùng K-1 phần để học và 1 phần để kiểm tra. CV chỉ nên làm trên tập train, tập test gốc phải tách khỏi mọi bước chọn model.

Data leakage (rò rỉ dữ liệu). Thông tin từ test lọt vào quá trình train, làm điểm đẹp giả. Pipeline của scikit-learn giúp tránh việc này vì đảm bảo mỗi bước chỉ fit trên đúng tập con dữ liệu. Rủi ro này có với hầu hết phép biến đổi, gồm StandardScaler, SimpleImputer, PCA.

Mất cân bằng lớp (class imbalance). Lớp dương hiếm thì accuracy đánh lừa, phải dùng metric khác (mục 4).

Regularization. Phạt model quá phức tạp (L1/L2, giới hạn độ sâu cây, `min_samples_leaf` …) để giảm overfitting.

Reproducibility. Cố định `random_state`, ghi lại phiên bản dữ liệu và siêu tham số.

3. Các mô hình

Mô hình	Ý tưởng	Mạnh	Yếu	Cần scale?
Logistic Regression	Đường ranh giới tuyến tính, đầu ra là xác suất	Nhanh, dễ giải thích, làm baseline tốt	Không bắt được quan hệ phi tuyến	Có
Decision Tree	Chuỗi câu hỏi if-else	Dễ hiểu, không cần scale	Rất dễ overfit nếu không giới hạn độ sâu	Không
Random Forest	Nhiều cây trên mẫu bootstrap + chọn ngẫu nhiên feature, bỏ phiếu (bagging)	Giảm variance, ổn định, ít cần chỉnh	Chậm hơn, khó giải thích hơn	Không
Gradient Boosting (XGBoost, LightGBM)	Cây sau sửa lỗi của cây trước	Thường mạnh nhất trên dữ liệu bảng	Nhiều siêu tham số, dễ overfit nếu chỉnh ẩu	Không
KNN	Nhìn K láng giềng gần nhất	Đơn giản	Chậm khi dự đoán, nhạy với thang đo	Có
Naive Bayes	Xác suất Bayes với giả định độc lập	Rất nhanh, tốt với văn bản	Giả định độc lập thường sai	Không bắt buộc
SVM	Tìm ranh giới có lề lớn nhất	Tốt khi dữ liệu ít chiều cao	Chậm với dữ liệu lớn	Có

Mô hình	Ý tưởng	Mạnh	Yếu	Cần scale?
K-Means, PCA	Gom cụm; giảm chiều (không giám sát)	Khám phá cấu trúc dữ liệu	Cần chọn K / số thành phần	Có

Mạng nơ-ron học sau, khi nền tảng ở trên đã vững.

4. Chỉ số đo lường

Confusion matrix là gốc của mọi metric phân loại: TP, FP, TN, FN.

- **Accuracy** = đúng / tổng. Không nên dùng làm metric chính trên bài toán mất cân bằng.
- **Precision** = TP / (TP + FP): trong số dự đoán dương, bao nhiêu đúng. Dùng khi báo động sai tốn kém.
- **Recall** = TP / (TP + FN): trong số dương thật, bắt được bao nhiêu. Dùng khi bỏ sót tốn kém.
- **F1**: trung bình điều hòa của precision và recall, chỉ cao khi cả hai cùng cao.
- **ROC-AUC**: đo khả năng xếp hạng, độc lập với ngưỡng. 0.5 là ngẫu nhiên, 1.0 là hoàn hảo.
- **PR-AUC (Average Precision)**: tập trung vào lớp dương. Khi dữ liệu mất cân bằng nặng, đường ROC có thể cho cái nhìn quá lạc quan, nên PR-AUC được đề xuất thay thế. Quy tắc thực dụng: lớp dương hiếm thì dùng PR-AUC, lớp tương đối cân bằng thì ROC-AUC. Nên báo cáo kèm tỷ lệ lớp dương làm mốc so sánh.
- **Hồi quy**: MAE (sai số trung bình tuyệt đối), MSE/RMSE (phạt nặng sai số lớn), R^2 (tỷ lệ phương sai giải thích được).

5. Cách biết model chạy thế nào

1. **So sánh với baseline** (`DummyClassifier`, hoặc Logistic Regression đơn giản). Model phức tạp không hơn baseline thì không đáng dùng.
2. **Khoảng cách train–test** (điểm train trừ điểm test trên cùng một metric, ví dụ F1 train so với F1 test). Cách đọc:

Triệu chứng	Chẩn đoán	Hướng xử lý
Train thấp, test thấp	Underfitting (bias cao)	Model phức tạp hơn, thêm feature, giảm regularization
Train cao, test thấp	Overfitting (variance cao)	Thêm dữ liệu, regularization, model đơn giản hơn
Cả hai cao, gần nhau	Ổn	Kiểm tra thêm leakage nếu cao bất thường

3. **Learning curve**: vẽ điểm train và validation theo số lượng mẫu. Hai đường thấp và hội tụ là high bias; train cao mà validation thấp hơn nhiều là high variance.
4. **Validation curve**: vẽ điểm theo một siêu tham số (như `max_depth`). Ví dụ điển hình là biểu đồ overfitting của Decision Tree khi tăng độ sâu.
5. **CV mean ± std**: std lớn nghĩa là kết quả không ổn định, không nên tin một lần chạy.
6. **Confusion matrix + phân tích lỗi**: xem model sai ở đâu, sai loại nào.
7. **Ngưỡng quyết định**: 0.5 chỉ là mặc định, hãy xem đường PR và chọn ngưỡng theo chi phí FP/FN.
8. **Chi phí tính toán**: thời gian huấn luyện và thời gian dự đoán (ví dụ ms trên 1000 dòng) cũng là một tiêu chí chọn model.

6. Những thứ không thể bỏ qua nếu học lại từ đầu

Đây là những thói quen và kiến thức dễ bị bỏ sót nhất; nên có ngay từ bài đầu tiên thay vì bổ sung về sau:

1. **Nền toán và công cụ**: xác suất–thống kê cơ bản (phân phối, kỳ vọng, phương sai), đại số tuyến tính (vector, ma trận), đạo hàm (để hiểu gradient descent); Python với NumPy, pandas, matplotlib, scikit-learn.
2. **Khám phá và làm sạch dữ liệu (EDA) trước khi huấn luyện**: kiểm tra giá trị thiếu (kể cả khi được mã hóa bằng ký tự như `?`), bản ghi trùng, cột trùng thông tin, cột mã định danh hoặc trọng số lấy mẫu không nên đưa vào như feature thường, và phân phối của target.
3. **Pipeline + ColumnTransformer** cho toàn bộ tiền xử lý (impute, one-hot, scale), để không bao giờ rò rỉ dữ liệu.
4. **Chia có phân tầng**: `stratify=y` khi chia và `StratifiedKFold` khi CV, đặc biệt với dữ liệu lệch lớp.

5. **Baseline**: model đoán lớp đa số, để biết accuracy 85% thật ra hơn được bao nhiêu.
6. **PR-AUC (average precision)** cạnh ROC-AUC, và ghi rõ tỷ lệ lớp dương làm mốc.
7. **Cùng một split và seed cho mọi model**, nếu không so sánh không công bằng. Chạy lặp nhiều seed rồi báo cáo mean ± std.
8. **Trọng số lớp và resampling** (ví dụ `class_weight='balanced'`) chỉ đổi cân bằng precision/recall chứ không làm model giỏi hơn; hãy giải thích được vì sao recall tăng còn precision giảm.
9. **Chỉnh ngưỡng** bên cạnh trọng số lớp, vì hai cách đều dịch chuyển điểm cân bằng precision–recall.
10. **Giải thích model có chừng mực**: feature importance mặc định của cây/Random Forest thiên về biến liên tục hoặc nhiều giá trị, nên dùng thêm *permutation importance* để kiểm chứng.
11. **Tập test chỉ dùng cuối cùng**, mọi lựa chọn siêu tham số dựa trên CV/validation.

Ba câu tự kiểm tra (trả lời được là đã nắm nền tảng): Vì sao dùng F1/PR-AUC thay vì accuracy trên dữ liệu lệch lớp? Vì sao Decision Tree dễ overfit còn Random Forest thì ít hơn? Làm sao biết kết quả không phải do may mắn khi chia dữ liệu? (gợi ý cho câu cuối: CV mean ± std).

7. Lỗi hay mắc, nguyên nhân gốc và cách sửa

Lỗi	Nguyên nhân gốc	Cách sửa
Scale/impute/chọn feature **trước khi** chia train/test	Thống kê (mean, std) tính cả từ dữ liệu test, thông tin test lọt vào train	Chia trước, fit transformer chỉ trên train, hoặc dùng Pipeline
Làm tiền xử lý ngoài vòng CV	Mỗi fold validation đã "thấy" thống kê của toàn bộ dữ liệu, sinh ra điểm CV cao giả; lỗi này rất phổ biến và thường không ai nhận ra	Đặt tiền xử lý **bên trong** Pipeline rồi đưa Pipeline vào `cross_val_score`
Dùng accuracy trên dữ liệu lệch lớp	Đoán toàn bộ là lớp đa số vẫn được điểm cao	Dùng precision/recall/F1/PR-AUC, xem confusion matrix

Lỗi	Nguyên nhân gốc	Cách sửa
Chỉ báo cáo ROC-AUC	TN nhiều làm ROC-AUC trông đẹp, che đi hiệu năng trên lớp hiếm	Báo thêm PR-AUC và tỷ lệ lớp dương
Tinh chỉnh siêu tham số trực tiếp trên test	Test bị "nhìn" nhiều lần, thành validation trá hình	Tách validation hoặc dùng CV trên train; test chỉ chạy một lần
Không `stratify`	Chia ngẫu nhiên có thể làm lớp hiếm lệch giữa train và test	`train_test_split(..., stratify=y)`, `StratifiedKFold`
Decision Tree không giới hạn độ sâu	Cây phát triển đến khi thuộc lòng từng mẫu train (điểm train ≈ 1, khoảng cách train–test lớn)	Giới hạn `max_depth`, `min_samples_leaf`, hoặc dùng Random Forest; chọn độ sâu bằng validation curve
Quên scale với LR/KNN/SVM	Thuật toán dựa trên khoảng cách hoặc gradient nhạy với thang đo	Thêm `StandardScaler` trong Pipeline (cây thì không cần)
Dùng `LabelEncoder` cho biến phân loại không có thứ tự rồi đưa vào model tuyến tính	Gán số 0,1,2 tạo ra thứ tự giả	One-hot encoding (`OneHotEncoder`)
Oversample (SMOTE) trước khi chia	Bản sao/mẫu tổng hợp của một điểm lọt vào cả train lẫn test	Chỉ resample **trong train** (dùng `imblearn.pipeline`)
So sánh model dựa trên một lần chạy	Điểm dao động theo cách chia và seed	CV nhiều fold, nhiều seed, báo mean ± std
Dùng ngưỡng 0.5 mặc định	Ngưỡng tối ưu phụ thuộc chi phí FP vs FN	Vẽ đường PR, chọn ngưỡng theo mục tiêu nghiệp vụ
Feature chứa thông tin của kết quả (target leakage)	Feature được tạo sau thời điểm cần dự đoán	Hỏi: lúc dự đoán thật, mình có biết cột này không? Điểm cao bất thường là dấu hiệu cần kiểm tra

Lỗi	Nguyên nhân gốc	Cách sửa
Bản ghi trùng nằm ở cả train và test	Model "nhớ" thay vì tổng quát hóa	Loại trùng lặp trước khi chia
Diễn giải feature importance như quan hệ nhân quả	Importance chỉ cho biết model dùng gì, không nói biến nào gây ra kết quả	Trình bày là "model dựa nhiều vào..." và dùng permutation importance để kiểm chứng

Thứ tự học lại gợi ý

1. Chia dữ liệu + overfitting/underfitting
2. Confusion matrix và metric
3. Logistic Regression và Decision Tree
4. CV + Pipeline + leakage
5. Random Forest / Boosting
6. Imbalance và chỉnh ngưỡng